

Bàn về tư tưởng phân tầng trong bài toán đường đi ngắn nhất

Lu Zihong

Trường Trung học số 7 Tuyền Châu, Phúc Kiến

Trại đông Tin học toàn quốc, 2008

Nguồn gốc: Bàn về tư tưởng phân tầng trong bài toán đường đi ngắn nhất

IOI China candidate team papers / 2008 / Day 1 / Lu Zihong / layered shortest paths.doc

Tóm tắt

Tư tưởng phân tầng và thuật toán đường đi ngắn nhất đều có phạm vi ứng dụng rộng. Bài viết phân tích một số đề thi tin học để trình bày hai cách dùng phân tầng trong bài toán đường đi ngắn nhất: dựng mô hình sao cho bài toán phức tạp trở thành một bài toán đường đi ngắn nhất thông thường, và khai thác cấu trúc tầng để tối ưu thuật toán. Các ví dụ gồm *Rescue*, *Fence*, *Cow Relay*, *Bicriterial Routing* và *Roads*.

Từ khóa: đường đi ngắn nhất; phân tầng; lý thuyết đồ thị.

1 Dẫn nhập

Bài toán đường đi ngắn nhất là một bài toán kinh điển của lý thuyết đồ thị. Trọng số cạnh không nhất thiết chỉ là khoảng cách hình học; trong nhiều mô hình, nó có thể là thời gian, chi phí hoặc một đại lượng cộng dồn tuyến tính dọc theo đường đi. Vì vậy đường đi ngắn nhất xuất hiện tự nhiên trong quy hoạch đô thị, định tuyến giao thông, tối ưu mạng và nhiều bài thi tin học.

Tư tưởng phân tầng cũng là một công cụ quan trọng. Ta gặp nó trong cách chia giai đoạn của quy hoạch động, trong các thuật toán luồng cực đại dựa trên đồ thị mức, và trong nhiều mô hình trạng thái khác. Khi kết hợp phân tầng với đường đi ngắn nhất, ta có thể đưa những ràng buộc tưởng như nằm ngoài đồ thị vào cấu trúc đỉnh và cạnh của một đồ thị mới.

Bài viết xét hai hướng chính. Phần đầu dùng phân tầng để dựng mô hình. Phần sau dùng tính chất của đồ thị phân tầng để tối ưu thuật toán.

2 Dùng phân tầng để dựng mô hình

Trong phần này, tác giả xét ba ví dụ: *Rescue*, *Fence* và *Cow Relay*. Điểm chung là bài toán ban đầu không phải một bài toán đường đi ngắn nhất chuẩn trên một đồ thị cố định, nhưng có thể trở thành như vậy nếu mỗi tầng biểu diễn một điều kiện bổ sung.

2.1 Ví dụ 1: Rescue

Một mê cung hình chữ nhật gồm N hàng và M cột. Hai ô kề nhau theo hướng bắc-nam hoặc đông-tây có thể thông nhau, có thể bị ngăn bởi một bức tường, hoặc có thể có một cánh cửa khóa. Các cửa được chia

thành P loại; cùng một loại dùng cùng một chìa khóa, các loại khác nhau dùng chìa khác nhau. Một số ô chứa chìa khóa. Mỗi bước đi sang ô kề tồn thời gian 1; thời gian nhất chìa và mở cửa được bỏ qua. Cần tìm thời gian ngắn nhất từ $(1, 1)$ tới (N, M) . Giới hạn là $N, M \leq 15, P \leq 10$.

Nếu bỏ qua cửa và chìa khóa, mỗi ô là một đỉnh, mỗi cặp ô thông nhau nối bởi một cạnh trọng số 1, và bài toán trở thành đường đi ngắn nhất chuẩn. Cửa và chìa khóa làm cho khả năng đi qua cạnh phụ thuộc vào trạng thái đã nhất chìa. Vì vậy ta chia đồ thị thành 2^P tầng, mỗi tầng ứng với một tập chìa khóa đang có.

Trong một tầng, các cạnh đi qua cửa chỉ tồn tại nếu tập chìa của tầng đó cho phép mở cửa. Với một ô chứa chìa, từ đỉnh tương ứng trong tầng hiện tại ta nối một cạnh trọng số 0 tới cùng ô ở tầng sau khi đã thêm chìa khóa ấy. Trên đồ thị mới, mọi trạng thái đã được biểu diễn bằng đỉnh, nên có thể chạy trực tiếp thuật toán đường đi ngắn nhất.

Vì mọi cạnh có trọng số 0 hoặc 1, ta có thể dùng BFS hai đầu hàng đợi hoặc một biến thể BFS cho trọng số 0/1. Độ phức tạp thời gian và bộ nhớ đều là

$$\mathcal{O}(2^P NM).$$

2.2 Ví dụ 2: Fence

Một điền trang có tòa nhà chính và nhiều công trình khác. Chủ điền trang muốn xây một hàng rào bao quanh tòa nhà chính, nhưng hàng rào không được đi quá gần bất kỳ công trình nào. Mỗi công trình được bao trong một hình chữ nhật cấm, các cạnh song song trục tọa độ; hàng rào cũng chỉ được gồm các đoạn song song trục. Cần tìm độ dài nhỏ nhất của một hàng rào hợp lệ bao quanh tòa nhà chính. Số hình chữ nhật $m \leq 100$, tọa độ không âm và không vượt quá 10000.

Mục tiêu là tối thiểu hóa một đại lượng cộng dồn tuyến tính, nên ta thử mô hình hóa bằng đường đi ngắn nhất. Trước hết xây một đồ thị một tầng: các đỉnh là các đỉnh của các hình chữ nhật. Nếu đoạn thẳng song song trục nối hai đỉnh không đi vào bên trong hình chữ nhật cấm nào, ta nối hai đỉnh ấy bằng một cạnh có trọng số bằng khoảng cách Manhattan. Mỗi chu trình trong đồ thị tương ứng với một phương án hàng rào, và độ dài chu trình là độ dài hàng rào.

Tuy nhiên, không phải chu trình nào cũng hợp lệ; nó phải bao quanh tòa nhà chính. Theo tiêu chuẩn hình học quen thuộc, một điểm nằm trong đa giác nếu một tia xuất phát từ điểm ấy cắt biên đa giác số lẻ lần. Vì vậy ta kẻ một tia từ tòa nhà chính và chỉ cần biết chu trình cắt tia ấy với số lần chẵn hay lẻ.

Ta tách đồ thị thành hai tầng, biểu diễn tính chẵn lẻ của số lần cắt tia. Nếu một cạnh của đồ thị một tầng cắt tia, cạnh đó nối hai tầng khác nhau; nếu không, nó nối trong cùng tầng. Khi đó tìm một chu trình bao quanh tòa nhà chính tương đương với tìm đường đi ngắn nhất giữa hai bản sao khác tầng của cùng một đỉnh. Lấy nhỏ nhất trên mọi đỉnh sẽ cho độ dài hàng rào tối ưu.

Nếu kiểm tra mọi cặp đỉnh bằng liệt kê hình chữ nhật, việc dựng cạnh tốn $\mathcal{O}(m^3)$. Chạy Dijkstra từ các đỉnh cần xét cũng có thể làm trong $\mathcal{O}(m^3)$. Bộ nhớ là $\mathcal{O}(m^2)$.

2.3 Ví dụ 3: Cow Relay

Farmer John chọn k con bò tham gia tiếp sức. Trang trại có n cánh đồng và m cạnh vô hướng có trọng số, biểu diễn thời gian đi qua. Các con bò lần lượt chạy từ cánh đồng 1 tới cánh đồng n ; khi một con tới đích, con sau lập tức xuất phát. Đường đi của hai con bất kỳ phải khác nhau. Cần tìm thời điểm con bò thứ k tới đích. Giới hạn là $k \leq 40, n < 800, m \leq 4000$.

Nhìn theo đồ thị, bài toán yêu cầu tìm đường đi thứ k ngắn nhất giữa hai đỉnh trong đồ thị có trọng số dương. Một cách trực tiếp là duy trì hàng đợi ưu tiên của các đường đi. Mỗi lần lấy đường đi ngắn nhất hiện có, mở rộng nó theo các cạnh ra từ đỉnh cuối, cho tới khi thu được k đường từ 1 tới n . Vì mỗi đỉnh

chỉ cần nhiều nhất k đường đi ứng viên để có thể thuộc một trong k đường tốt nhất tới n , số đường được mở rộng không vượt quá kn . Độ phức tạp có thể viết là

$$\mathcal{O}(km + kn \log(kn)).$$

Cách này đúng nhưng chưa đủ đẹp.

Bản gốc đưa ra một cách nhìn khác. Gọi P là tập mọi đường đi từ 1 tới n , và p_i là đường đi ngắn thứ i . Nếu sau khi tìm được p_i ta loại bỏ đường đó khỏi tập đường đi, thì đường ngắn nhất còn lại chính là p_{i+1} . Vấn đề là phải thực hiện thao tác “loại một đường đi” mà không phá hỏng toàn bộ đồ thị.

Ta thêm hai đỉnh s, t , cạnh $(s, 1)$ và (n, t) có trọng số 0. Mỗi đường từ 1 tới n tương ứng một đường từ s tới t . Sau khi tìm đường ngắn nhất p_{st1} , sao chép một phần các đỉnh và cạnh của đồ thị. Với mỗi cạnh (i, j) không thuộc p_{st1} , ta thêm cạnh chuyển từ bản gốc của i sang bản sao của j . Với các cạnh thuộc p_{st1} , không thêm cạnh chuyển như vậy. Các cạnh trong tầng sao chép giữ nguyên trọng số.

Nếu xem mỗi đỉnh gốc và bản sao của nó là cùng một đỉnh, mọi đường từ s tới t' trong đồ thị mới đều là một đường cũ từ s tới t , nhưng không thể là đúng p_{st1} , vì đường ấy bị chặn tại cạnh đầu tiên cần chuyển tầng. Ngược lại, mọi đường khác p_{st1} đều có cạnh đầu tiên khác với p_{st1} , nên tại cạnh đó có thể chuyển sang tầng sao chép và tiếp tục tới t' . Vì vậy tập đường từ s tới t' chính là tập cũ sau khi loại p_{st1} . Đường ngắn nhất trong tập này là đường ngắn thứ hai.

Lặp quá trình trên sẽ thu được các đường ngắn tiếp theo. Không cần sao chép toàn bộ đồ thị ở mỗi bước: nếu v_k là đỉnh đầu tiên trên p_{st1} có bậc vào lớn hơn 1, chỉ cần sao chép đoạn từ v_k trở đi. Những đỉnh không nằm trên đường hoặc nằm trước v_k không tạo ra đường chuyển tầng hữu ích từ s .

Do đồ thị mới chỉ thêm đỉnh và cạnh vào đồ thị cũ, cây đường đi ngắn nhất của phần cũ không bị ảnh hưởng. Khi tìm đường đi mới, ta có thể chỉ duy trì cây đường đi ngắn nhất trên các đỉnh vừa sao chép, thay vì chạy lại toàn bộ. Nếu dùng SPFA cho cây đường đi ngắn nhất, mỗi vòng có thể xem là $\mathcal{O}(m)$, tổng thời gian $\mathcal{O}(km)$, bộ nhớ $\mathcal{O}(kn + m)$.

2.4 Nhận xét về dựng mô hình

Ba ví dụ trên dùng phân tầng theo ba cách khác nhau. Trong *Rescue*, tầng biểu diễn trạng thái chìa khóa, tức điều kiện quyết định một số cạnh có đi được hay không. Trong *Fence*, tầng biểu diễn tính chẵn lẻ của số lần cắt một tia, tức điều kiện để chu trình bao quanh tòa nhà chính. Trong *Cow Relay*, tầng và cạnh chuyển tầng dùng để loại bỏ một đường đi đã chọn, làm cho mỗi tầng có một tập đường đi khác nhau.

Bản chất là: khi một đồ thị một tầng không biểu diễn đủ các điều kiện của bài toán, ta sao chép nó thành nhiều tầng, mỗi tầng tương ứng một điều kiện. Việc phân tầng làm tăng kích thước đồ thị, nhưng nếu khai thác được cấu trúc của các tầng thì phần tăng thêm thường vẫn chấp nhận được.

3 Dùng phân tầng để tối ưu thuật toán

Phần trước dùng phân tầng để biến bài toán thành đường đi ngắn nhất. Phần này xét chiều ngược lại: khi đã có đồ thị phân tầng, chính cấu trúc tầng có thể giúp giảm độ phức tạp.

3.1 Ví dụ 4: Bicriterial Routing

Mạng đường thu phí của Byteland có các đường hai chiều. Mỗi đường có thời gian đi và phí phải trả. Với cùng điểm xuất phát và đích, một tuyến đường A tốt hơn tuyến B nếu A nhanh hơn mà phí không cao hơn, hoặc phí thấp hơn mà không chậm hơn. Một tuyến được gọi là tối ưu theo hai tiêu chí nếu không có

tuyến nào tốt hơn nó. Cần đếm số cặp (phí, thời gian) tối ưu từ s tới e . Giới hạn: $n \leq 100$, $m \leq 300$, mỗi phí và thời gian không vượt quá 100.

Khác với đường đi ngắn nhất thường, mỗi cạnh có hai trọng số. Ta chọn phí làm chỉ số tầng. Nếu c là phí lớn nhất của một cạnh, một đường tối ưu không cần có tổng phí vượt quá nc trong mô hình của bài. Do đó ta tách đồ thị thành $nc + 1$ tầng, tầng q biểu diễn đã dùng tổng phí q . Một cạnh có phí w và thời gian t nối từ tầng q sang tầng $q + w$, với trọng số đường đi là t .

Đồ thị mới có $\mathcal{O}(nc)$ tầng, mỗi tầng có n đỉnh và m cạnh kiểu tương ứng, nên số đỉnh là $\mathcal{O}(n^2c)$, số cạnh là $\mathcal{O}(nmc)$. Vì thời gian không âm, có thể chạy Dijkstra trên toàn đồ thị. Cách trực tiếp tốn khoảng

$$\mathcal{O}(ncm \log(n^2c)).$$

Nhưng đồ thị có tính chất quan trọng: phí không âm, nên cạnh chỉ đi trong cùng tầng hoặc từ tầng thấp sang tầng cao hơn. Tầng phí cao không ảnh hưởng ngược lại tầng phí thấp. Vì vậy thay vì chạy một Dijkstra trên toàn bộ đồ thị, ta xử lý từng tầng theo thứ tự phí tăng dần; mỗi tầng chỉ cần một lần đường đi ngắn nhất trên n đỉnh. Nếu dùng heap nhị phân, mỗi tầng tốn $\mathcal{O}(m \log n)$, tổng cộng

$$\mathcal{O}(ncm \log n).$$

Sau khi có thời gian tốt nhất cho từng mức phí, ta quét các cặp (phí, thời gian) để loại những cặp bị trội và đếm các cặp tối ưu.

3.2 Ví dụ 5: Roads

Có N thành phố và các đường một chiều. Mỗi đường có chiều dài và phí. Bob có nhiều nhất k đồng, xuất phát từ thành phố 1, muốn tới thành phố N nhanh nhất trong phạm vi chi phí cho phép. Giới hạn: $k \leq 10000$, $n \leq 100$, $m \leq 10000$, chiều dài mỗi đường không vượt quá 100, phí mỗi đường không âm và không vượt quá 100.

Mô hình đầu tiên giống bài trước: tách đồ thị thành $k + 1$ tầng theo số tiền đã dùng. Một cạnh có phí w nối từ tầng q sang tầng $q + w$, trọng số là chiều dài đường. Vì chiều dài dương, có thể dùng Dijkstra. Nếu cài đặt đơn giản bằng mảng cho đồ thị khá dày, độ phức tạp là

$$\mathcal{O}(k(kn^2 + m)).$$

Tương tự *Bicriterial Routing*, vì phí không âm nên có thể xử lý các tầng theo thứ tự phí tăng dần, giảm còn

$$\mathcal{O}(k(n^2 + m)).$$

Tuy vậy, bài này có thêm điều kiện mà mô hình theo phí chưa dùng hết: chiều dài đường là số nguyên dương và có chặn nhỏ. Ta có thể đổi trục phân tầng sang chiều dài và dùng quy hoạch động. Gọi $f[i, j]$ là số tiền ít nhất cần dùng để tới thành phố j bằng một đường có tổng chiều dài đúng i . Khi có cạnh từ j_0 tới j với chiều dài ℓ và phí w , ta cập nhật

$$f[i, j] = \min(f[i, j], f[i - \ell, j_0] + w).$$

Điều kiện đầu là $f[0, 1] = 0$, các trạng thái khác ban đầu là vô hạn. Đáp án là i nhỏ nhất sao cho $f[i, N] \leq k$. Nếu L là chiều dài cạnh lớn nhất, cách này có độ phức tạp $\mathcal{O}(nLm)$, tốt hơn trong bối cảnh giới hạn của đề.

3.3 Nhận xét về tối ưu

Đồ thị phân tầng thường có nhiều phần giống nhau giữa các tầng. Ta không nhất thiết phải lưu toàn bộ đỉnh và cạnh một cách tường minh; nhiều cạnh có thể được sinh khi cần. Ngoài ra, nếu cạnh chỉ đi từ tầng thấp sang tầng cao, các tầng tạo thành một thứ tự topo theo chiều tầng. Khi đó ta có thể xử lý từng tầng, làm giảm kích thước hàng đợi ưu tiên hoặc số vòng lặp của Bellman-Ford và SPFA.

Trong trường hợp cấu trúc trong cùng một tầng cũng đơn giản, đường đi ngắn nhất có thể được thay bằng quy hoạch động. So với Dijkstra, quy hoạch động bỏ được thao tác trên hàng đợi ưu tiên, nên hằng số nhỏ hơn và cài đặt rõ hơn.

4 Kết luận

Bài viết trình bày hai vai trò của phân tầng trong bài toán đường đi ngắn nhất. Khi dựng mô hình, phân tầng giúp biểu diễn các yếu tố khó đặt vào một đồ thị đơn: trạng thái chìa khóa, tính chẵn lẻ của giao điểm, hay tập đường đi còn lại sau khi loại một đường. Khi tối ưu thuật toán, cấu trúc tầng giúp xử lý theo thứ tự tự nhiên, tiết kiệm không gian và thời gian, thậm chí thay đường đi ngắn nhất bằng quy hoạch động.

Phân tầng không phải một công thức máy móc. Cần chọn đại lượng làm tầng sao cho nó thật sự nắm giữ yếu tố gây khó của bài toán; đồng thời phải khai thác được tính chất của tầng để phân phình ra của đồ thị không làm thuật toán mất hiệu quả.

Tác giả cảm ơn cô Xie Shuiying đã hướng dẫn và hỗ trợ, đồng thời cảm ơn huấn luyện viên Liu Rujia đã đưa ra nhiều ý kiến quý báu.

Tài liệu tham khảo

1. Yu Yuanming, *Thuật toán đường đi ngắn nhất và ứng dụng*, bài viết đội tuyển quốc gia Trung Quốc năm 2006.
2. Xiao Tian, *Tư tưởng đồ thị phân tầng và ứng dụng trong thi tin học*, bài viết đội tuyển quốc gia Trung Quốc năm 2004.
3. Baltic Olympiad in Informatics 2002/2007 official solutions.
4. E. Q. V. Martins và J. L. E. Santos, “A new shortest paths ranking algorithm”, *Investigacao Operacional*, 20(1):47–62, 2000.

Ghi chú về phụ lục nguồn

Tài liệu gốc kèm các đề bài đầy đủ của *Rescue*, *Fence*, *Cow Relay*, *Bicriterial Routing* và *Roads*. Phần phụ lục trong DOC có nhiều đoạn tiếng Anh bị hỏng mã hóa; bản dịch này giữ nội dung cần cho mạch phân tích trong các ví dụ ở trên và mô tả phụ lục bằng lời thay vì sao chép phần hỏng.